Escape the Network Graph Data

Copy the starter data. Run each test setup separately from a fresh copy.

Nodes and names

```
nodes = ["A", "B", "C", "D", "E", "F"]
location_names = [
    "Server Room",
    "Laboratory",
    "Storage",
    "Security Hub",
    "Maintenance Tunnel",
    "Exit"
]
```

Matching indices describe one location: index 0 is A, Server Room; index 5 is F, Exit.

Starter adjacency matrix

```
matrix = [
    [None,    4,    2, None, None, None],  # A
    [   4, None, None,    5,    3, None],  # B
    [   2, None, None,    1, None, None],  # C
    [None,    5,    1, None,    1,    7],  # D
    [None,    3, None,    1, None,    2],  # E
    [None, None, None,    7,    2, None]   # F
]
```

matrix[row][column] is the edge cost: A–C costs 2 in both directions. None means absent; 0 is a valid cost.

Fresh search data

```
query_matrix = [row[:] for row in matrix]

visited = [False] * len(nodes)
predecessors = [None] * len(nodes)
distances = [float("inf")] * len(nodes)
```

Copy every row before blocking an edge. Reset search records for each algorithm; set the start distance to 0 for Dijkstra. Use alphabetical neighbour order.

Separate test setups

Query A→F unless stated otherwise. Use test_nodes/test_names for expanded graphs and reset search records to their new length.

Expensive direct edge

```
query_matrix = [row[:] for row in matrix]
query_matrix[0][5] = 20
query_matrix[5][0] = 20
```

BFS: A→F, cost 20. Dijkstra: A→C→D→E→F, cost 6.

Blocked connection

```
query_matrix = [row[:] for row in matrix]
query_matrix[3][4] = None  # Block D to E
query_matrix[4][3] = None  # Block E to D
```

Cheapest route: A→B→E→F, cost 9. Original graph unchanged.

Unreachable exit

```
test_nodes = nodes + ["G"]
test_names = location_names + ["Isolated Exit"]
query_matrix = [row[:] + [None] for row in matrix]
query_matrix.append([None] * 7)
```

Query A→G: no route; all searches terminate.

Dead-end branch

```
test_nodes = nodes + ["G", "H"]
test_names = location_names + ["Side Corridor", "Locked Store"]
query_matrix = [row[:] + [None, None] for row in matrix]
query_matrix.append([None] * 8)
query_matrix.append([None] * 8)
query_matrix[1][6] = query_matrix[6][1] = 3  # B to G
query_matrix[6][7] = query_matrix[7][6] = 2  # G to H
```

H is a dead end. Dijkstra still finds cost 6; search order affects expansions.

Further checks

Starter BFS: A→B→D→F, cost 16. A→A: [A], 0 edges, cost 0. Reject unknown labels and negative weights. Set A –B to 0 both ways: it must remain usable.